

Phase_3_Cross_Validation_SII_Slovenia_demo

November 4, 2025

1 Slovenia Solvency reports Table S.02.01.02; Phase 3 Cross-Validation

This script is a continuation of `Phase_2_Processing_Slovenia_SII_demo`.

The purpose of this script is to perform the third and final automated stage of the S.02.01.02 transcribing. Cross-Validation takes the generated table and performs a series of internal consistency checks. First on an aggregate level and then for each company in the dataset. The functions return a `None` if the check is passed or the fields that are compared if the check fails. This allows the user to quickly identify the mistakes in the transcribing.

The way we operate this notebook is that we first look at the summary of all companies. If the entire array is true (all tests pass for all companies), then the process moves into the final phase. If not, the user looks further down through the notebook at individual companies. Where a test fails, the relevant fields are displayed. The user then compares the pdf with the values displayed. If any corrections are necessary, the user does them above the test in code. In this way, when the script is run again, the tests pass.

1.1 Description of the process

The process of extraction is performed in 5 phases:

1.1.1 Phase 0: Find the reports and identify the relevant tables.

- 1) Identify the new SFCR report and save it into the folder `Input`.
- 2) Identify the pages where the tables of interest are.
- 3) Compile the map of the company run in the `master_list.csv`.

1.1.2 Phase 1: Run the Extraction script.

The script performs the following steps (with slight modifications depending on the table format):

- 1) Save the page with the table into a separate folder `Single_pdf`.
- 2) Use either a Python package or specialized LLM to create a digital equivalent of the table.
- 3) Fix the systemic errors that prevent the table from being saved as `DataFrame`.
- 4) Save the `DataFrame` into the `Output` folder.

1.1.3 Phase 2: Run the Processing script.

The script applies fixes to the `DataFrame` to make the numbers closer to the reported numbers. It joins all the tables into a single dataset and saves it into the `Dirty_Combined` folder.

1.1.4 Phase 3: Run the Cross-Validation script (this script).

Applies a series of tests that check for the internal consistency between the numbers. Flags potential errors. After the individual fixes are applied, it saves the table into the Cleaner_Combined folder.

1.1.5 Phase 4: Final modifications to the table and a manual inspection.

1.1.6 Tolerance

```
[357]: eps = 1.5
```

Necessary packages

```
[358]: import pandas as pd
import numpy as np
```

1.1.7 Functions

Functions that start with “check_” implement a single internal consistency check. Comparing two or more numbers if they match.

```
[359]: def check_02_01_02_1(data:pd.DataFrame, eps:float, col:str) -> list[bool, pd.
↳DataFrame, float]:
    """
    R0100 = R0110 + R0120

    """

    lhs = data.loc["R0100",col]
    rhs = data.loc["R0110",col] + data.loc["R0120",col]
    diff = abs(lhs-rhs)
    if diff< eps:
        return True, None, diff
    else:
        return False, pd.DataFrame(data = [data.loc["R0110",col], data.
↳loc["R0120",col], data.loc["R0100",col]], index=["R0110", "R0120","R0100"],
↳columns = ["TEST_1"]), diff
```

```
[360]: def check_02_01_02_2(data:pd.DataFrame, eps:float, col:str) -> list[bool, pd.
↳DataFrame, float]:
    """
    R0130 = R0140 + R0150 + R0160 + R0170

    """

    lhs = data.loc["R0130",col]
    rhs = data.loc["R0140",col] + data.loc["R0150",col]+ data.loc["R0160",col]+
↳data.loc["R0170",col]
    diff = abs(lhs-rhs)
```

```

    if diff < eps:
        return True, None, diff
    else:
        diagnostics = [data.loc["R0140",col], data.loc["R0150",col], data.
↪loc["R0160",col], data.loc["R0170",col], data.loc["R0130",col]]
        diagnostics_index = ["R0140", "R0150", "R0160", "R0170", "R0130"]
        return False, pd.DataFrame(data = diagnostics, index=diagnostics_index,
↪columns = ["TEST_2"]), diff

```

```

[361]: def check_02_01_02_3(data:pd.DataFrame, eps:float, col:str) -> list[bool, pd.
↪DataFrame, float]:
    """
    R0070 = R0080 + R0090 + R0100 + R0130 + R0180 + R0190 + R0200 + R0210

    """

    lhs = data.loc["R0070",col]
    rhs = data.loc["R0080",col] + data.loc["R0090",col]+ data.loc["R0100",col]+
↪data.loc["R0130",col] + data.loc["R0180",col]+ data.loc["R0190",col]+ data.
↪loc["R0200",col]+ data.loc["R0210",col]
    diff = abs(lhs-rhs)
    if diff < eps:
        return True, None, diff
    else:
        diagnostics = [data.loc["R0080",col], data.loc["R0090",col], data.
↪loc["R0100",col], data.loc["R0130",col],
                        data.loc["R0180",col], data.loc["R0190",col], data.
↪loc["R0200",col], data.loc["R0210",col], data.loc["R0070",col]]

        diagnostics_index = ["R0080", "R0090", "R0100", "R0130", "R0180",
↪"R0190", "R0200", "R0210", "R0070"]
        return False, pd.DataFrame(data = diagnostics, index=diagnostics_index,
↪columns = ["TEST_3"]), diff

```

```

[362]: def check_02_01_02_4(data:pd.DataFrame, eps:float, col:str) -> list[bool, pd.
↪DataFrame, float]:
    """
    R0230 = R0240 + R0250 + R0260

    """

    lhs = data.loc["R0230",col]
    rhs = data.loc["R0240",col] + data.loc["R0250",col]+ data.loc["R0260",col]
    diff = abs(lhs-rhs)
    if diff < eps:
        return True, None, diff

```

```

    else:
        diagnostics = [data.loc["R0240",col], data.loc["R0250",col], data.
↪loc["R0260",col], data.loc["R0230",col]]
        diagnostics_index = ["R0240", "R0250", "R0260", "R0230"]
        return False, pd.DataFrame(data = diagnostics, index=diagnostics_index,
↪columns = ["TEST_4"]), diff

```

```

[363]: def check_02_01_02_5(data:pd.DataFrame, eps:float, col:str) -> list[bool, pd.
↪DataFrame, float]:
    """
     $R0270 = R0280 + R0310 + R0340$ 

    """

    lhs = data.loc["R0270",col]
    rhs = data.loc["R0280",col] + data.loc["R0310",col]+ data.loc["R0340",col]
    diff = abs(lhs-rhs)
    if diff< eps:
        return True, None, diff
    else:
        diagnostics = [data.loc["R0280",col], data.loc["R0310",col], data.
↪loc["R0340",col], data.loc["R0270",col]]
        diagnostics_index = ["R0280", "R0310", "R0340", "R0270"]
        return False, pd.DataFrame(data = diagnostics, index=diagnostics_index,
↪columns = ["TEST_5"]), diff

```

```

[364]: def check_02_01_02_6(data:pd.DataFrame, eps:float, col:str) -> list[bool, pd.
↪DataFrame, float]:
    """
     $R0310 = R0320 + R0330$ 

    """

    lhs = data.loc["R0310",col]
    rhs = data.loc["R0320",col] + data.loc["R0330",col]
    diff = abs(lhs-rhs)
    if diff< eps:
        return True, None, diff
    else:
        diagnostics = [data.loc["R0320",col], data.loc["R0330",col], data.
↪loc["R0310",col]]
        diagnostics_index = ["R0320", "R0330", "R0310"]
        return False, pd.DataFrame(data = diagnostics, index=diagnostics_index,
↪columns = ["TEST_6"]), diff

```

```
[365]: def check_02_01_02_7(data:pd.DataFrame, eps:float, col:str) -> list[bool, pd.
↳DataFrame, float]:
    """
    R0280 = R0290 + R0300

    """

    lhs = data.loc["R0280",col]
    rhs = data.loc["R0290",col] + data.loc["R0300",col]
    diff = abs(lhs-rhs)
    if diff< eps:
        return True, None, diff
    else:
        diagnostics = [data.loc["R0290",col], data.loc["R0300",col], data.
↳loc["R0280",col]]
        dagnostics_index = ["R0290", "R0300", "R0280"]
        return False, pd.DataFrame(data = diagnostics, index=dagnostics_index,
↳columns = ["TEST_7"]), diff
```

```
[366]: def check_02_01_02_8(data:pd.DataFrame, eps:float, col:str) -> list[bool, pd.
↳DataFrame, float]:
    """
    R0500 = R0030 + R0040 + R0050 + R0060 + R0070 + R0220 + R0230 +
    R0270 + R0350 + R0360 + R0370 + R0380 + R0390 + R0400 + R0410 +
↳R0420

    """

    lhs = data.loc["R0500",col]
    rhs = data.loc["R0030",col] + data.loc["R0040",col] + data.loc["R0050",col]
↳+ data.loc["R0060",col] + data.loc["R0070",col] + data.loc["R0220",col] +
↳data.loc["R0230",col] + data.loc["R0270",col] + data.loc["R0350",col] + data.
↳loc["R0360",col]+ data.loc["R0370",col]+ data.loc["R0380",col]+ data.
↳loc["R0390",col]+ data.loc["R0400",col]+ data.loc["R0410",col]+ data.
↳loc["R0420",col]
    diff = abs(lhs-rhs)
    if diff< eps:
        return True, None, diff
    else:
        diagnostics = [data.loc["R0030",col], data.loc["R0040",col], data.
↳loc["R0050",col], data.loc["R0060",col], data.loc["R0070",col], data.
↳loc["R0220",col], data.loc["R0230",col], data.loc["R0270",col], data.
↳loc["R0350",col], data.loc["R0360",col], data.loc["R0370",col], data.
↳loc["R0380",col], data.loc["R0390",col], data.loc["R0400",col], data.
↳loc["R0410",col], data.loc["R0420",col], data.loc["R0500",col]]
```

```

        diagnostics_index = ["R0030", "R0040", "R0050", "R0060", "R0070",
↪ "R0220", "R0230", "R0270", "R0350", "R0360", "R0370", "R0380", "R0390",
↪ "R0400", "R0410", "R0420", "R0500"]
        return False, pd.DataFrame(data = diagnostics, index=diagnostics_index,
↪ columns = ["TEST_8"]), diff

```

```

[367]: def check_02_01_02_9(data:pd.DataFrame, eps:float, col:str) -> list[bool, pd.
↪ DataFrame, float]:
        """
        R0510 = R0520 + R0560

        """

        lhs = data.loc["R0510",col]
        rhs = data.loc["R0520",col] + data.loc["R0560",col]
        diff = abs(lhs-rhs)
        if diff< eps:
            return True, None, diff
        else:
            diagnostics = [data.loc["R0520",col], data.loc["R0560",col], data.
↪ loc["R0510",col]]
            diagnostics_index = ["R0520", "R0560", "R0510"]
            return False, pd.DataFrame(data = diagnostics, index=diagnostics_index,
↪ columns = ["TEST_9"]), diff

```

```

[368]: def check_02_01_02_10(data:pd.DataFrame, eps:float, col:str) -> list[bool, pd.
↪ DataFrame, float]:
        """
        R0520 = R0530 + R0540 + R0550

        """

        lhs = data.loc["R0520",col]
        rhs = data.loc["R0530",col] + data.loc["R0540",col]+ data.loc["R0550",col]
        diff = abs(lhs-rhs)
        if diff< eps:
            return True, None, diff
        else:
            diagnostics = [data.loc["R0530",col], data.loc["R0540",col], data.
↪ loc["R0550",col], data.loc["R0520",col]]
            diagnostics_index = ["R0530", "R0540", "R0550", "R0520"]
            return False, pd.DataFrame(data = diagnostics, index=diagnostics_index,
↪ columns = ["TEST_10"]), diff

```

```

[369]: def check_02_01_02_11(data:pd.DataFrame, eps:float, col:str) -> list[bool, pd.
↪ DataFrame, float]:
        """

```

```

R0560 = R0570 + R0580 + R0590

"""

lhs = data.loc["R0560",col]
rhs = data.loc["R0570",col] + data.loc["R0580",col]+ data.loc["R0590",col]
diff = abs(lhs-rhs)
if diff< eps:
    return True, None, diff
else:
    diagnostics = [data.loc["R0570",col], data.loc["R0580",col], data.
↳loc["R0590",col], data.loc["R0560",col]]
    diagnostics_index = ["R0570", "R0580", "R0590", "R0560"]
    return False, pd.DataFrame(data = diagnostics, index=diagnostics_index,↳
↳columns = ["TEST_11"]), diff

```

```

[370]: def check_02_01_02_12(data:pd.DataFrame, eps:float, col:str) -> list[bool, pd.
↳DataFrame, float]:
    """
    R0600 = R0610 + R0650

    """

    lhs = data.loc["R0600",col]
    rhs = data.loc["R0610",col] + data.loc["R0650",col]
    diff = abs(lhs-rhs)
    if diff< eps:
        return True, None, diff
    else:
        diagnostics = [data.loc["R0610",col], data.loc["R0650",col], data.
↳loc["R0600",col]]
        diagnostics_index = ["R0610", "R0650", "R0600"]
        return False, pd.DataFrame(data = diagnostics, index=diagnostics_index,↳
↳columns = ["TEST_12"]), diff

```

```

[371]: def check_02_01_02_13(data:pd.DataFrame, eps:float, col:str) -> list[bool, pd.
↳DataFrame, float]:
    """
    R0610 = R0620 + R0630 + R0640

    """

    lhs = data.loc["R0610",col]
    rhs = data.loc["R0620",col] + data.loc["R0630",col]+ data.loc["R0640",col]
    diff = abs(lhs-rhs)
    if diff< eps:
        return True, None, diff

```

```

    else:
        diagnostics = [data.loc["R0620",col], data.loc["R0630",col], data.
↪loc["R0640",col], data.loc["R0610",col]]
        diagnostics_index = ["R0620", "R0630", "R0640", "R0610"]
        return False, pd.DataFrame(data = diagnostics, index=diagnostics_index,
↪columns = ["TEST_13"]), diff

```

```

[372]: def check_02_01_02_14(data:pd.DataFrame, eps:float, col:str) -> list[bool, pd.
↪DataFrame, float]:
    """
     $R0650 = R0660 + R0670 + R0680$ 

    """

    lhs = data.loc["R0650",col]
    rhs = data.loc["R0660",col] + data.loc["R0670",col]+ data.loc["R0680",col]
    diff = abs(lhs-rhs)
    if diff< eps:
        return True, None, diff
    else:
        diagnostics = [data.loc["R0660",col], data.loc["R0670",col], data.
↪loc["R0680",col], data.loc["R0650",col]]
        diagnostics_index = ["R0660", "R0670", "R0680", "R0650"]
        return False, pd.DataFrame(data = diagnostics, index=diagnostics_index,
↪columns = ["TEST_14"]), diff

```

```

[373]: def check_02_01_02_15(data:pd.DataFrame, eps:float, col:str) -> list[bool, pd.
↪DataFrame, float]:
    """
     $R0690 = R0700 + R0710 + R0720$ 

    """

    lhs = data.loc["R0690",col]
    rhs = data.loc["R0700",col] + data.loc["R0710",col]+ data.loc["R0720",col]
    diff = abs(lhs-rhs)
    if diff< eps:
        return True, None, diff
    else:
        diagnostics = [data.loc["R0700",col], data.loc["R0710",col], data.
↪loc["R0720",col], data.loc["R0690",col]]
        diagnostics_index = ["R0700", "R0710", "R0720", "R0690"]
        return False, pd.DataFrame(data = diagnostics, index=diagnostics_index,
↪columns = ["TEST_15"]), diff

```



```
[374]: def check_02_01_02_16(data:pd.DataFrame, eps:float, col:str) -> list[bool, pd.
↳DataFrame, float]:
    """
    R0850 = R0860 + R0870

    """

    lhs = data.loc["R0850",col]
    rhs = data.loc["R0860",col] + data.loc["R0870",col]
    diff = abs(lhs-rhs)
    if diff< eps:
        return True, None, diff
    else:
        diagnostics = [data.loc["R0860",col], data.loc["R0870",col], data.
↳loc["R0850",col]]
        dagnostics_index = ["R0860", "R0870", "R0850"]
        return False, pd.DataFrame(data = diagnostics, index=diagnostics_index,↳
↳columns = ["TEST_16"]), diff
```

```
[375]: def check_02_01_02_17(data:pd.DataFrame, eps:float, col:str) -> list[bool, pd.
↳DataFrame, float]:
    """
    R1000 = R0500 - R0900

    """

    lhs = data.loc["R1000",col]
    rhs = data.loc["R0500",col] - data.loc["R0900",col]
    diff = abs(lhs-rhs)
    if diff< eps:
        return True, None, diff
    else:
        diagnostics = [data.loc["R0500",col], data.loc["R0900",col], data.
↳loc["R1000",col]]
        diagnostics_index = ["R0500", "R0900", "R1000"]
        return False, pd.DataFrame(data = diagnostics, index=diagnostics_index,↳
↳columns = ["TEST_17"]), diff
```

```
[376]: def run_check_diagnostics(table: pd.DataFrame, function, col_name: str, eps:↳
↳float) -> pd.DataFrame:
    """
    Runs a single check provided as a function for every column of a dataframe.

    """
    results = pd.DataFrame(data=[], columns=["COMPANY_NAME",col_name])
    for col in table.columns:
        res, diag, observed_tol = function(table, eps=eps, col=col)
```

```

        result_tmp = pd.DataFrame(data=[[col, res]], columns =
↪ ["COMPANY_NAME", col_name])
        results = pd.concat([results, result_tmp])
        results[results == True] = ""

    return results

```

1.2 Dirty table

Uploads to memory the dirty data table from the previous phase.

```

[377]: dirty_table = pd.read_csv("Dirty_Combined/Table_slo_S02.csv", header=0,
↪ index_col=0)

```

```

[378]: dirty_table = dirty_table.loc[dirty_table.index.notna()]

```

```

[379]: dirty_table.drop(index=["CODE"], inplace=True)

```

```

[380]: dirty_table.drop(columns=["MASTER"], inplace=True)

```

```

[381]: dirty_table = dirty_table.astype(float).fillna(0)

```

1.3 High-level overview of quality

Performs the checks for all companies at the same time. Assessing the overall quality of the table before looking at individual companies.

```

[382]: results_1 = run_check_diagnostics(dirty_table, check_02_01_02_1, "TEST_1", eps)
        results_1 = results_1.set_index("COMPANY_NAME")

```

```

[383]: results_2 = run_check_diagnostics(dirty_table, check_02_01_02_2, "TEST_2", eps)
        results_2 = results_2.set_index("COMPANY_NAME")

```

```

[384]: results_3 = run_check_diagnostics(dirty_table, check_02_01_02_3, "TEST_3", eps)
        results_3 = results_3.set_index("COMPANY_NAME")

```

```

[385]: results_4 = run_check_diagnostics(dirty_table, check_02_01_02_4, "TEST_4", eps)
        results_4 = results_4.set_index("COMPANY_NAME")

```

```

[386]: results_5 = run_check_diagnostics(dirty_table, check_02_01_02_5, "TEST_5", eps)
        results_5 = results_5.set_index("COMPANY_NAME")

```

```

[387]: results_6 = run_check_diagnostics(dirty_table, check_02_01_02_6, "TEST_6", eps)
        results_6 = results_6.set_index("COMPANY_NAME")

```

```

[388]: results_7 = run_check_diagnostics(dirty_table, check_02_01_02_7, "TEST_7", eps)
        results_7 = results_7.set_index("COMPANY_NAME")

```

```

[389]: results_8 = run_check_diagnostics(dirty_table, check_02_01_02_8, "TEST_8", eps)
       results_8 = results_8.set_index("COMPANY_NAME")

[390]: results_9 = run_check_diagnostics(dirty_table, check_02_01_02_9, "TEST_9", eps)
       results_9 = results_9.set_index("COMPANY_NAME")

[391]: results_10 = run_check_diagnostics(dirty_table, check_02_01_02_10, "TEST_10",
      ↪eps)
       results_10 = results_10.set_index("COMPANY_NAME")

[392]: results_11 = run_check_diagnostics(dirty_table, check_02_01_02_11, "TEST_11",
      ↪eps)
       results_11 = results_11.set_index("COMPANY_NAME")

[393]: results_12 = run_check_diagnostics(dirty_table, check_02_01_02_12, "TEST_12",
      ↪eps)
       results_12 = results_12.set_index("COMPANY_NAME")

[394]: results_13 = run_check_diagnostics(dirty_table, check_02_01_02_13, "TEST_13",
      ↪eps)
       results_13 = results_13.set_index("COMPANY_NAME")

[395]: results_14 = run_check_diagnostics(dirty_table, check_02_01_02_14, "TEST_14",
      ↪eps)
       results_14 = results_14.set_index("COMPANY_NAME")

[396]: results_15 = run_check_diagnostics(dirty_table, check_02_01_02_15, "TEST_15",
      ↪eps)
       results_15 = results_15.set_index("COMPANY_NAME")

[397]: results_16 = run_check_diagnostics(dirty_table, check_02_01_02_16, "TEST_16",
      ↪eps)
       results_16 = results_16.set_index("COMPANY_NAME")

[398]: results_17 = run_check_diagnostics(dirty_table, check_02_01_02_17, "TEST_17",
      ↪eps)
       results_17 = results_17.set_index("COMPANY_NAME")

[399]: overall_summary = pd.
      ↪concat([results_1,results_2,results_3,results_4,results_5,results_6,results_7,results_8,res
      ↪
      ↪results_12,results_13,results_14,results_15,results_16,results_17], axis=1)

[400]: display(overall_summary)

```

```

          TEST_1 TEST_2 TEST_3 TEST_4 TEST_5 TEST_6 TEST_7 TEST_8 TEST_9 \
COMPANY_NAME
MODRA

```

TRIGLAV								
PRVA						False		
GENERALI								
VZAJEMNA								
GRAWE							False	
SAVA								

	TEST_10	TEST_11	TEST_12	TEST_13	TEST_14	TEST_15	TEST_16	TEST_17
COMPANY_NAME								
MODRA			False		False			
TRIGLAV			False	False	False			False
PRVA			False		False	False		False
GENERALI								
VZAJEMNA								
GRAWE						False		
SAVA								

1.4 Manual overwrites

If a check is failing, modify the numbers at the begining of the section until the tests are passed.
Note that sometimes small errors are due to rounding.

1.5 Modra zavarovalnica d.d.

```
[402]: unique_code = "MODRA"
[403]: dirty_table.loc["R0650",unique_code] = 382508.46
[404]: result,diagnostics, diff = check_02_01_02_1(dirty_table, eps, unique_code)
      diagnostics
[405]: result,diagnostics, diff = check_02_01_02_2(dirty_table, eps, unique_code)
      diagnostics
[406]: result,diagnostics, diff = check_02_01_02_3(dirty_table, eps, unique_code)
      diagnostics
[407]: result,diagnostics, diff = check_02_01_02_4(dirty_table, eps, unique_code)
      diagnostics
[408]: result,diagnostics, diff = check_02_01_02_5(dirty_table, eps, unique_code)
      diagnostics
[409]: result,diagnostics, diff = check_02_01_02_6(dirty_table, eps, unique_code)
      diagnostics
[410]: result,diagnostics, diff = check_02_01_02_7(dirty_table, eps, unique_code)
      diagnostics
```

```
[411]: result,diagnostics, diff = check_02_01_02_8(dirty_table, eps, unique_code)
diagnostics

[412]: result,diagnostics, diff = check_02_01_02_9(dirty_table, eps, unique_code)
diagnostics

[413]: result,diagnostics, diff = check_02_01_02_10(dirty_table, eps, unique_code)
diagnostics

[414]: result,diagnostics, diff = check_02_01_02_11(dirty_table, eps, unique_code)
diagnostics

[415]: result,diagnostics, diff = check_02_01_02_12(dirty_table, eps, unique_code)
diagnostics

[416]: result,diagnostics, diff = check_02_01_02_13(dirty_table, eps, unique_code)
diagnostics

[417]: result,diagnostics, diff = check_02_01_02_14(dirty_table, eps, unique_code)
diagnostics

[418]: result,diagnostics, diff = check_02_01_02_15(dirty_table, eps, unique_code)
diagnostics

[419]: result,diagnostics, diff = check_02_01_02_16(dirty_table, eps, unique_code)
diagnostics

[420]: result,diagnostics, diff = check_02_01_02_17(dirty_table, eps, unique_code)
diagnostics
```

1.6 TRIGLAV

```
[421]: unique_code = "TRIGLAV"

[422]: dirty_table.loc["R0650",unique_code] = 699614
dirty_table.loc["R0600",unique_code] = 699761
dirty_table.loc["R0640",unique_code] = 0
dirty_table.loc["R0660",unique_code] = 0
dirty_table.loc["R0670",unique_code] = 678631
dirty_table.loc["R0680",unique_code] = 20983
dirty_table.loc["R0900",unique_code] = 2471275
dirty_table.loc["R1000",unique_code] = 1044800

[423]: result,diagnostics, diff = check_02_01_02_1(dirty_table, eps, unique_code)
diagnostics

[424]: result,diagnostics, diff = check_02_01_02_2(dirty_table, eps, unique_code)
diagnostics
```

```
[425]: result,diagnostics, diff = check_02_01_02_3(dirty_table, eps, unique_code)
diagnostics

[426]: result,diagnostics, diff = check_02_01_02_4(dirty_table, eps, unique_code)
diagnostics

[427]: result,diagnostics, diff = check_02_01_02_5(dirty_table, eps, unique_code)
diagnostics

[428]: result,diagnostics, diff = check_02_01_02_6(dirty_table, eps, unique_code)
diagnostics

[429]: result,diagnostics, diff = check_02_01_02_7(dirty_table, eps, unique_code)
diagnostics

[430]: result,diagnostics, diff = check_02_01_02_8(dirty_table, eps, unique_code)
diagnostics

[431]: result,diagnostics, diff = check_02_01_02_9(dirty_table, eps, unique_code)
diagnostics

[432]: result,diagnostics, diff = check_02_01_02_10(dirty_table, eps, unique_code)
diagnostics

[433]: result,diagnostics, diff = check_02_01_02_11(dirty_table, eps, unique_code)
diagnostics

[434]: result,diagnostics, diff = check_02_01_02_12(dirty_table, eps, unique_code)
diagnostics

[435]: result,diagnostics, diff = check_02_01_02_13(dirty_table, eps, unique_code)
diagnostics

[436]: result,diagnostics, diff = check_02_01_02_14(dirty_table, eps, unique_code)
diagnostics

[437]: result,diagnostics, diff = check_02_01_02_15(dirty_table, eps, unique_code)
diagnostics

[438]: result,diagnostics, diff = check_02_01_02_16(dirty_table, eps, unique_code)
diagnostics

[439]: result,diagnostics, diff = check_02_01_02_17(dirty_table, eps, unique_code)
diagnostics
```

1.7 Prva osebna zavarovalnica, d.d.

```
[440]: unique_code = "PRVA"

[441]: dirty_table.loc["R0500",unique_code] = 51250
dirty_table.loc["R0610",unique_code] = 59
dirty_table.loc["R0650",unique_code] = 3040
dirty_table.loc["R0630",unique_code] = -408
dirty_table.loc["R0640",unique_code] = 466
dirty_table.loc["R0710",unique_code] = 14810

[442]: result,diagnostics, diff = check_02_01_02_1(dirty_table, eps, unique_code)
diagnostics

[443]: result,diagnostics, diff = check_02_01_02_2(dirty_table, eps, unique_code)
diagnostics

[444]: result,diagnostics, diff = check_02_01_02_3(dirty_table, eps, unique_code)
diagnostics

[445]: result,diagnostics, diff = check_02_01_02_4(dirty_table, eps, unique_code)
diagnostics

[446]: result,diagnostics, diff = check_02_01_02_5(dirty_table, eps, unique_code)
diagnostics

[447]: result,diagnostics, diff = check_02_01_02_6(dirty_table, eps, unique_code)
diagnostics

[448]: result,diagnostics, diff = check_02_01_02_7(dirty_table, eps, unique_code)
diagnostics

[449]: result,diagnostics, diff = check_02_01_02_8(dirty_table, eps, unique_code)
diagnostics

[450]: result,diagnostics, diff = check_02_01_02_9(dirty_table, eps, unique_code)
diagnostics

[451]: result,diagnostics, diff = check_02_01_02_10(dirty_table, eps, unique_code)
diagnostics

[452]: result,diagnostics, diff = check_02_01_02_11(dirty_table, eps, unique_code)
diagnostics

[453]: result,diagnostics, diff = check_02_01_02_12(dirty_table, eps, unique_code)
diagnostics
```

```
[454]: result,diagnostics, diff = check_02_01_02_13(dirty_table, eps, unique_code)
diagnostics
```

```
[455]: result,diagnostics, diff = check_02_01_02_14(dirty_table, eps, unique_code)
diagnostics
```

```
[456]: result,diagnostics, diff = check_02_01_02_15(dirty_table, eps, unique_code)
diagnostics
```

```
[457]: result,diagnostics, diff = check_02_01_02_16(dirty_table, eps, unique_code)
diagnostics
```

```
[458]: result,diagnostics, diff = check_02_01_02_17(dirty_table, eps, unique_code)
diagnostics
```

1.8 GENERALI

```
[459]: unique_code = "GENERALI"
```

```
[460]: result,diagnostics, diff = check_02_01_02_1(dirty_table, eps, unique_code)
diagnostics
```

```
[461]: result,diagnostics, diff = check_02_01_02_2(dirty_table, eps, unique_code)
diagnostics
```

```
[462]: result,diagnostics, diff = check_02_01_02_3(dirty_table, eps, unique_code)
diagnostics
```

```
[463]: result,diagnostics, diff = check_02_01_02_4(dirty_table, eps, unique_code)
diagnostics
```

```
[464]: result,diagnostics, diff = check_02_01_02_5(dirty_table, eps, unique_code)
diagnostics
```

```
[465]: result,diagnostics, diff = check_02_01_02_6(dirty_table, eps, unique_code)
diagnostics
```

```
[466]: result,diagnostics, diff = check_02_01_02_7(dirty_table, eps, unique_code)
diagnostics
```

```
[467]: result,diagnostics, diff = check_02_01_02_8(dirty_table, eps, unique_code)
diagnostics
```

```
[468]: result,diagnostics, diff = check_02_01_02_9(dirty_table, eps, unique_code)
diagnostics
```

```
[469]: result,diagnostics, diff = check_02_01_02_10(dirty_table, eps, unique_code)
diagnostics
```



```
[470]: result,diagnostics, diff = check_02_01_02_11(dirty_table, eps, unique_code)
diagnostics
```

```
[471]: result,diagnostics, diff = check_02_01_02_12(dirty_table, eps, unique_code)
diagnostics
```

```
[472]: result,diagnostics, diff = check_02_01_02_13(dirty_table, eps, unique_code)
diagnostics
```

```
[473]: result,diagnostics, diff = check_02_01_02_14(dirty_table, eps, unique_code)
diagnostics
```

```
[474]: result,diagnostics, diff = check_02_01_02_15(dirty_table, eps, unique_code)
diagnostics
```

```
[475]: result,diagnostics, diff = check_02_01_02_16(dirty_table, eps, unique_code)
diagnostics
```

```
[476]: result,diagnostics, diff = check_02_01_02_17(dirty_table, eps, unique_code)
diagnostics
```

1.9 Vzajemna d.d.

```
[477]: unique_code = "VZAJEMNA"
```

```
[478]: dirty_table.loc["R0510",unique_code] = 13908
```

```
[479]: result,diagnostics, diff = check_02_01_02_1(dirty_table, eps, unique_code)
diagnostics
```

```
[480]: result,diagnostics, diff = check_02_01_02_2(dirty_table, eps, unique_code)
diagnostics
```

```
[481]: result,diagnostics, diff = check_02_01_02_3(dirty_table, eps, unique_code)
diagnostics
```

```
[482]: result,diagnostics, diff = check_02_01_02_4(dirty_table, eps, unique_code)
diagnostics
```

```
[483]: result,diagnostics, diff = check_02_01_02_5(dirty_table, eps, unique_code)
diagnostics
```

```
[484]: result,diagnostics, diff = check_02_01_02_6(dirty_table, eps, unique_code)
diagnostics
```

```
[485]: result,diagnostics, diff = check_02_01_02_7(dirty_table, eps, unique_code)
diagnostics
```

```
[486]: result,diagnostics, diff = check_02_01_02_8(dirty_table, eps, unique_code)
diagnostics

[487]: result,diagnostics, diff = check_02_01_02_9(dirty_table, eps, unique_code)
diagnostics

[488]: result,diagnostics, diff = check_02_01_02_10(dirty_table, eps, unique_code)
diagnostics

[489]: result,diagnostics, diff = check_02_01_02_11(dirty_table, eps, unique_code)
diagnostics

[490]: result,diagnostics, diff = check_02_01_02_12(dirty_table, eps, unique_code)
diagnostics

[491]: result,diagnostics, diff = check_02_01_02_13(dirty_table, eps, unique_code)
diagnostics

[492]: result,diagnostics, diff = check_02_01_02_14(dirty_table, eps, unique_code)
diagnostics

[493]: result,diagnostics, diff = check_02_01_02_15(dirty_table, eps, unique_code)
diagnostics

[494]: result,diagnostics, diff = check_02_01_02_16(dirty_table, eps, unique_code)
diagnostics

[495]: result,diagnostics, diff = check_02_01_02_17(dirty_table, eps, unique_code)
diagnostics
```

1.10 Grawe zavarovalnica d.d.

```
[496]: unique_code = "GRAWE"

[497]: dirty_table.loc["R0510",unique_code] = 28104
dirty_table.loc["R0720",unique_code] = 1950

[498]: result,diagnostics, diff = check_02_01_02_1(dirty_table, eps, unique_code)
diagnostics

[499]: result,diagnostics, diff = check_02_01_02_2(dirty_table, eps, unique_code)
diagnostics

[500]: result,diagnostics, diff = check_02_01_02_3(dirty_table, eps, unique_code)
diagnostics

[501]: result,diagnostics, diff = check_02_01_02_4(dirty_table, eps, unique_code)
diagnostics
```

```
[502]: result,diagnostics, diff = check_02_01_02_5(dirty_table, eps, unique_code)
diagnostics
```

```
[503]: result,diagnostics, diff = check_02_01_02_6(dirty_table, eps, unique_code)
diagnostics
```

```
[504]: result,diagnostics, diff = check_02_01_02_7(dirty_table, eps, unique_code)
diagnostics
```

```
[505]: result,diagnostics, diff = check_02_01_02_8(dirty_table, eps, unique_code)
diagnostics
```

```
[506]: result,diagnostics, diff = check_02_01_02_9(dirty_table, eps, unique_code)
diagnostics
```

```
[507]: result,diagnostics, diff = check_02_01_02_10(dirty_table, eps, unique_code)
diagnostics
```

```
[508]: result,diagnostics, diff = check_02_01_02_11(dirty_table, eps, unique_code)
diagnostics
```

```
[509]: result,diagnostics, diff = check_02_01_02_12(dirty_table, eps, unique_code)
diagnostics
```

```
[510]: result,diagnostics, diff = check_02_01_02_13(dirty_table, eps, unique_code)
diagnostics
```

```
[511]: result,diagnostics, diff = check_02_01_02_14(dirty_table, eps, unique_code)
diagnostics
```

```
[512]: result,diagnostics, diff = check_02_01_02_15(dirty_table, eps, unique_code)
diagnostics
```

```
[513]: result,diagnostics, diff = check_02_01_02_16(dirty_table, eps, unique_code)
diagnostics
```

```
[514]: result,diagnostics, diff = check_02_01_02_17(dirty_table, eps, unique_code)
diagnostics
```

1.11 Zavarovalnica Sava, d.d.

```
[515]: unique_code = "SAVA"
```

```
[516]: result,diagnostics, diff = check_02_01_02_1(dirty_table, eps, unique_code)
diagnostics
```

```
[517]: result,diagnostics, diff = check_02_01_02_2(dirty_table, eps, unique_code)
diagnostics
```

```
[518]: result,diagnostics, diff = check_02_01_02_3(dirty_table, eps, unique_code)
      diagnostics

[519]: result,diagnostics, diff = check_02_01_02_4(dirty_table, eps, unique_code)
      diagnostics

[520]: result,diagnostics, diff = check_02_01_02_5(dirty_table, eps, unique_code)
      diagnostics

[521]: result,diagnostics, diff = check_02_01_02_6(dirty_table, eps, unique_code)
      diagnostics

[522]: result,diagnostics, diff = check_02_01_02_7(dirty_table, eps, unique_code)
      diagnostics

[523]: result,diagnostics, diff = check_02_01_02_8(dirty_table, eps, unique_code)
      diagnostics

[524]: result,diagnostics, diff = check_02_01_02_9(dirty_table, eps, unique_code)
      diagnostics

[525]: result,diagnostics, diff = check_02_01_02_10(dirty_table, eps, unique_code)
      diagnostics

[526]: result,diagnostics, diff = check_02_01_02_11(dirty_table, eps, unique_code)
      diagnostics

[527]: result,diagnostics, diff = check_02_01_02_12(dirty_table, eps, unique_code)
      diagnostics

[528]: result,diagnostics, diff = check_02_01_02_13(dirty_table, eps, unique_code)
      diagnostics

[529]: result,diagnostics, diff = check_02_01_02_14(dirty_table, eps, unique_code)
      diagnostics

[530]: result,diagnostics, diff = check_02_01_02_15(dirty_table, eps, unique_code)
      diagnostics

[531]: result,diagnostics, diff = check_02_01_02_16(dirty_table, eps, unique_code)
      diagnostics

[532]: result,diagnostics, diff = check_02_01_02_17(dirty_table, eps, unique_code)
      diagnostics
```

1.12 Master table after cross checks

Saves the final data table in the folder `Clearer_Combined`.

```
[533]: dirty_table.to_csv("Cleaner_Combined/Table_slo_S02.csv")
```